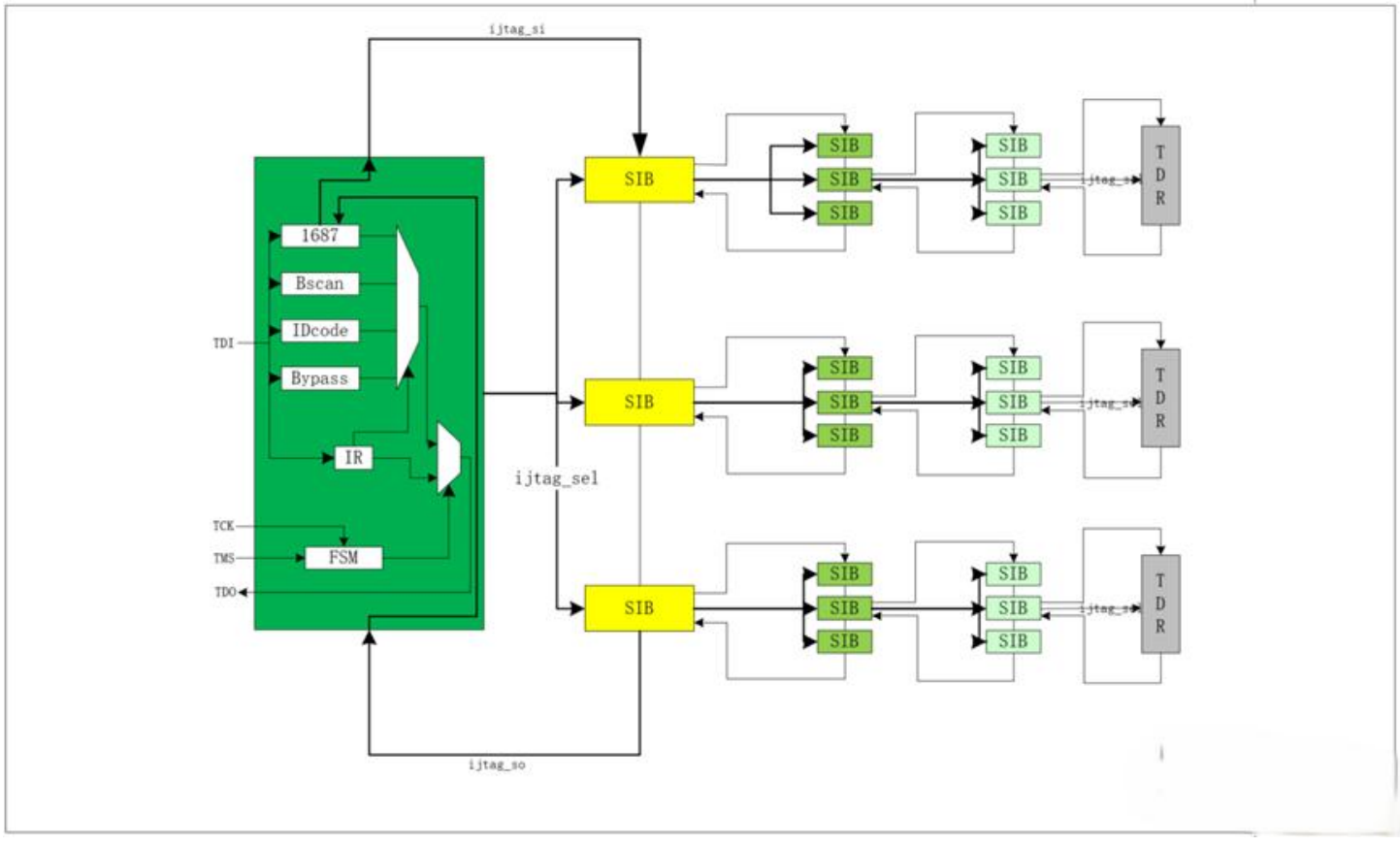
MBIST TEST.

1687 ijtag network



1687 network uses switch sib to select the lower-level TDR , realizing multi-level management and being more flexible.

Mbist introduction.

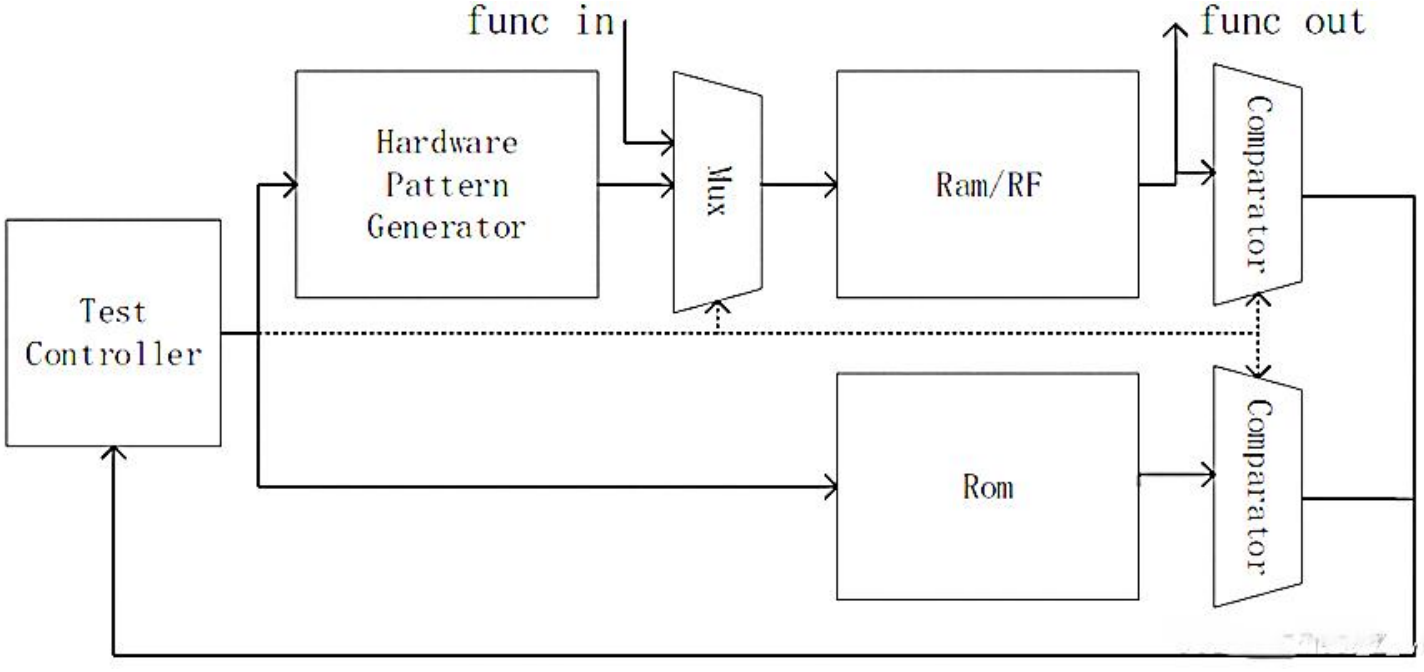
MBIST is short for Memory Build-In-Self Test. The test vectors for the memory are not generated by an external test machine (ATE: Auto -Test-Equipment), but are automatically generated by the inserted memory test logic and compiled. In the MBIST test, you only need to issue test instructions from the machine through the JTAG standard interface and obtain the test results from the TDO interface.

The memory integrates many storage units in a very small area, and the testing cost for ATE is too high, and it is not suitable to inject test vectors for the memory from the outside.
Memory has many units that need to be tested, but they are distributed regularly. EDA algorithms can be used to generate test vectors in batches.

The safety of automotive chips requires that the chips remain under online testing during operation (that is, they do not rely on testing machines and can be integrated at the board level, in the chip, or called by software for testing at any time) to detect and troubleshoot at any time.
Therefore, we can summarize the advantages of built-in self-test: because the test vectors are generated by internal logic, the corresponding modules can work together with the tested memory under the internal high-speed functional clock, without the need to move the test vectors from the machine's slow clock, which can save a lot of test time; on the other hand, the comparison verification is also done by the internal logic, and the test machine only needs to collect the test results, which can also greatly reduce the test time. The cost is that the built-in self-test logic occupies a high area and does not have the conditions for freely configuring or changing the test vectors.

Mbist Architecture.

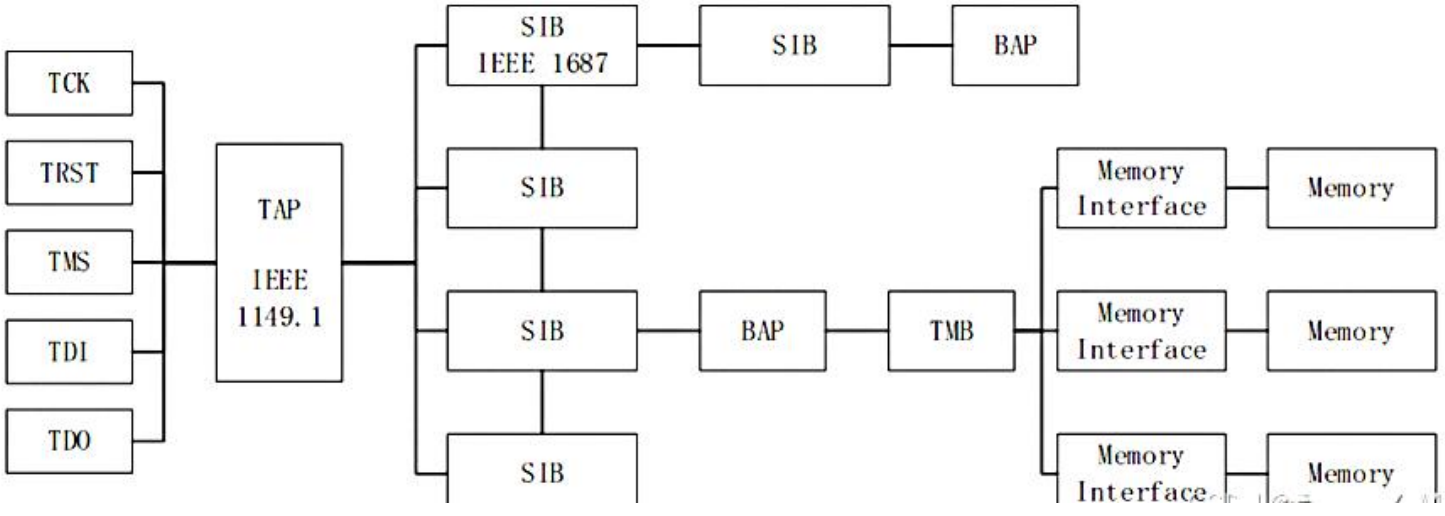
The MBIST test framework consists of a test controller, hardware vector generation, and comparator.



When the test control module receives the instruction to start the test, it will first switch the input and output of the memory to the test mode, and start the hardware vector generation module to generate and give test stimuli, and calculate the expected output value of the memory. After the memory receives the test vector, it will perform write / read / enable operations at intervals, traversing the write / read function of each bit unit under all addresses . Finally, the read value output by the Q terminal will be compared

with the expected value calculated by the test control module, and the correct result will be fed back to the test control module.

Tessent MBIST Logical Structure.



TAP: Test Access port

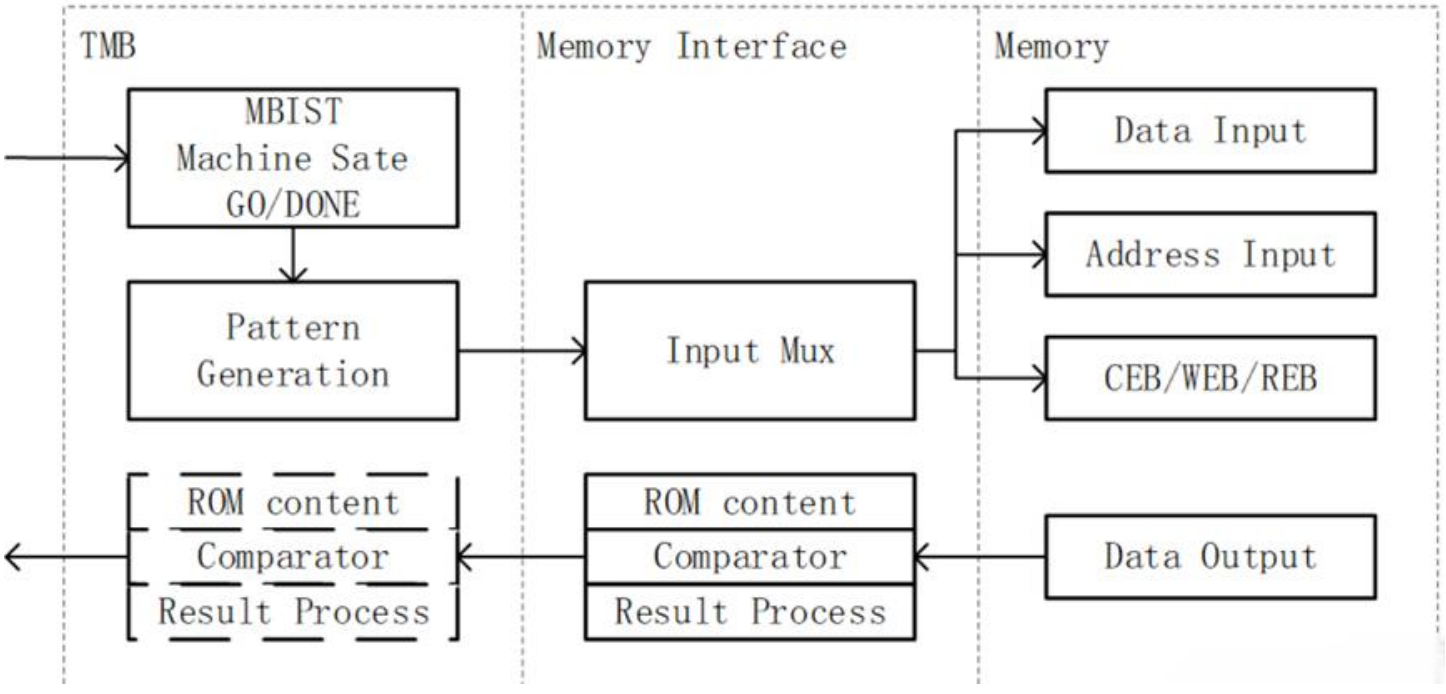
BAP: BIST Access port

SIB: Segment Insertion Bit

TMB: Tessent Memory BIST Controller

The Tessent Mbist structure is based on the IEEE1687 standard (TAP is IEEE1149.1). The function of TAP is to obtain external test instructions from the five groups of JTAG ports, convert them to the IJTAG scan chain, and shift them to the module behind it; SIB can turn on or off the corresponding IJTAG scan chain. The opening of the scan chain means that the memory of this part enters the test state; BAP acts as an interface for SIB to send to MBIST Controller; MBIST controller contains state machine logic and vector generation logic for memory test control; Memory interface includes MUX for selecting input and output; and the result comparison circuit, including the expected value of ROM, can be located in MBIST Controller (excellent area, poor timing and congestion) or in Memory interface.

How to test mem in Mbist?



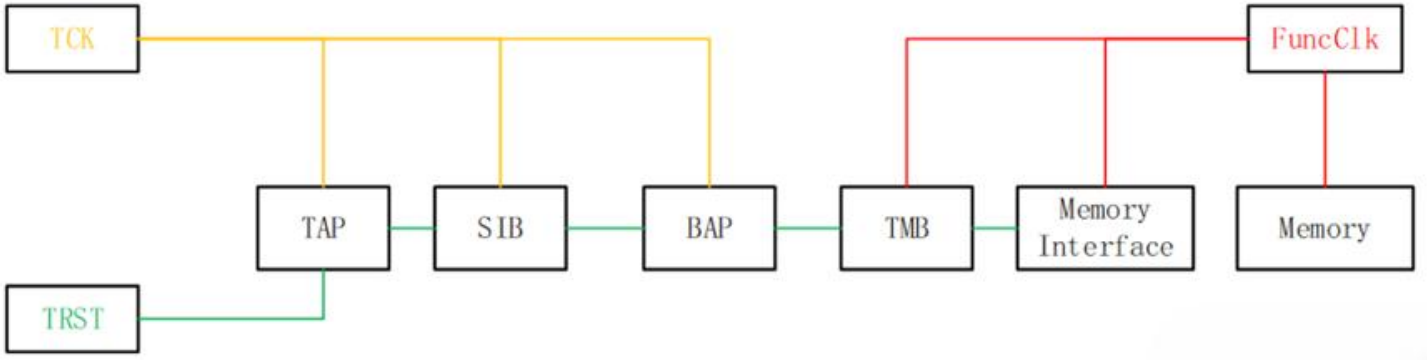
The key indication signals in the MBIST test are RUN , GO , and DONE signals, which are given by the MBIST Controller in combination with the comparison results of the state machine and the comparator. The RUN signal indicates that the current Controller and its subordinate memories have entered the test state (MBIST Mode); the GO=1 signal indicates that the test vector is input to the subordinate memories and the test begins; GO=0 indicates that the test comparator reports Fail ; and the DONE=1 signal indicates that the MBIST test has ended. The GO and DONE signals are combined to determine the test status of the memories under the current controller .

GO has not been pulled high or DONE has not been pulled high: Controller execution is incorrect;
After GO is pulled high, DONE has not been pulled high: Memory test is being executed and no error has occurred;
GO falls after being pulled high but before DONE is pulled high: Memory test fails;
After GO is pulled high, DONE is pulled high: the Memory test passes correctly.
The GO signal can have the following output options depending on user needs:

All test signals are aggregated into a GO signal and shifted out, indicating only the designed MBIST test result;
Each memory corresponds to a GO signal shifted out, which can specifically indicate which memory is faulty;

Each Comparator corresponds to a GO signal shifted out, which can be refined to find out which bit of the Memory output is faulty;
If you want to find out which BitCell is faulty, this is not possible by default and you need to call Tessent's Diagnosis Feature.

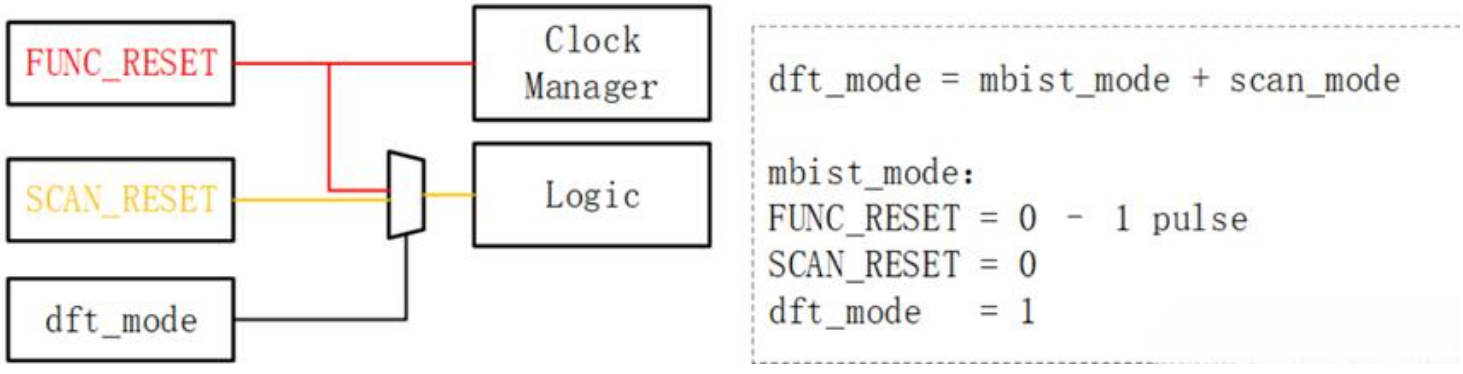
How to plan the Mbist clock ?



As shown above, it is a schematic diagram of the clock and reset signals of Tessent MBIST logic. The reset signal is provided to TMB and Memory Interface by TRST , TAP , SIB , and BAP step by step . Only when needed, the next level will get the reset signal from the previous level to start working. In terms of clock signals, TAP , SIB , and BAP are all on the IJTAG scan chain, so they work under the slow clock (TCK) of the machine ; TMB and Memory Interface are directly hung under the clock source (FuncClk) of the memory to be tested . The vector generation and test process are all working under the high-speed clock to save test time and facilitate full-speed testing of memory .

Therefore, we can clearly understand that the MBIST logic can work normally and does not rely on the original reset of the system, but it is necessary to ensure that the clock source of the memory is accurate and effective . The clock gating (Clock Gate), clock selection (Clock Mux), clock division (Clock Divider) and crystal clock (PLL) in the clock network may be correct and effective in functional mode; but if the clock path of the memory in test mode is not considered in the code design, it is very likely that after the circuit is switched to MBIST mode, due to signal configuration errors or part of the functional logic cannot work, the clock path is blocked or the clock frequency does not match the requirements of full-speed testing. As a result, the front-end designers have to rework and modify the code, resulting in unnecessary iterations of the project. Or the DFT engineer can perform ECO on the circuit structure by himself , but the problem with doing so is that the code is difficult to meet the complete standard, and the ECO modification work needs to repeatedly occupy the working time of the DFT engineer, which will cause duplication of work every time the project is updated, different DFT

engineers (such as the customer), and new and old projects are reused. In addition, considering the low-power design, the func clock may be specially separated for use by mbist. In this way, when testing mem, the func clock can be turned off, and only the logic related to mbist can be toggled to achieve the effect of reducing power consumption. Moreover, the low-power design is also related to the number of mems mounted under the mbist controller and the number of mems under a step. The power consumption issue needs to be considered during the design, and the number of mems that can be tested at the same time should be maximized within a reasonable range.



In addition, although the MBIST logic does not rely on the original reset of the system, the clock-related logic, such as the clock frequency division circuit in our CPU and the clock management module in the SOC platform, still needs to work normally to provide the correct clock. Therefore, the clock-related logic needs to get a reset rising edge signal that is the same as the functional mode . For other irrelevant logic, it is necessary to ensure that its reset is always low during the MBIST test to ensure that the irrelevant logic will not generate excessive power consumption due to signal flipping during the test. The reset processing method in the SOC platform is shown in the figure above .

If it is helpful to you reward me, it will encourage me to post good DFT contents.

Thanks for your reward.